École d'automne IA, Alsace Tech

Hackathon voix du client

TP plaintes : regrouper pour repérer les incidents

Fiche étudiants : la chaîne principale, puis la chaîne de comparaison



Objectif : transformer des plaintes écrites en plusieurs langues en problèmes nommés, puis repérer ceux qui explosent sur une marque et une période, sans jamais lire ni sortir un texte confidentiel.

1. Le problème en une page

Une entreprise agroalimentaire reçoit chaque mois des centaines de plaintes de consommateurs, dans plusieurs pays et plusieurs langues. Chaque plainte est un court texte libre, le **verbatim**, accompagné d'un numéro de cas, de la marque (numérotée), du segment d'activité et de la période. **Personne n'a classé ces textes.**

Quand plusieurs consommateurs décrivent le même problème, peut-on les regrouper et voir si ce problème se concentre sur une marque et une période ?

Comme le courrier d'une mairie. Si dix habitants écrivent la même semaine que l'eau du robinet a un goût bizarre, c'est sans doute une canalisation, pas dix coïncidences. Mais si la moitié de la ville se plaint du froid en janvier, ce n'est pas une panne : c'est l'hiver. Et si mille lettres identiques arrivent le même jour, c'est une pétition. Le TP apprend à faire la différence.

1.1 Trois phénomènes qui se ressemblent, et qu'il faut séparer

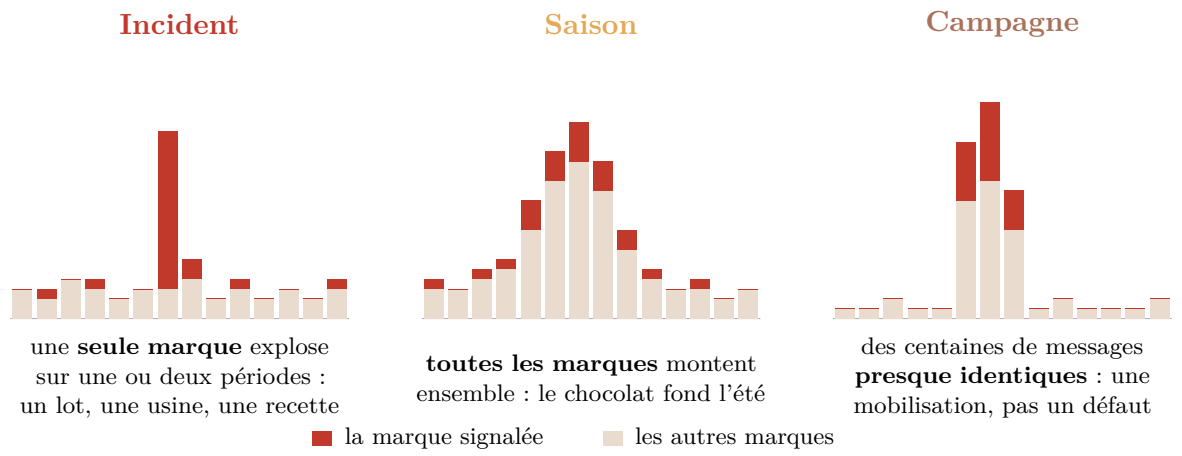


Schéma inventé : une barre par période, pour un même problème.

Incident candidat

un problème en **excès** sur une case (marque, période), et cette marque porte au moins la moitié du pic. **C'est ce qu'on cherche.**

Pic partagé

un problème en excès sur une période, mais réparti entre les marques : une saison, un phénomène général.

Campagne

un groupe dont la plupart des messages ont une jumelle **presque identique**.

Il n'y a pas d'étiquette. On ne prédit rien : on **regroupe**. C'est de l'apprentissage non supervisé, et toute la difficulté est de montrer que les groupes trouvés veulent dire quelque chose. Le notebook travaille sur la **base réelle**. Les figures de cette fiche viennent d'une **base fictive** de même forme, pour montrer à quoi ressemblent les résultats sans montrer de données confidentielles.

2. Deux grandes parties, dans cet ordre

Le travail se fait en deux temps. D'abord la **chaîne principale** : un seul outil, de la lecture du fichier jusqu'au tableau de bord. Ensuite seulement, une fois qu'elle marche, la **chaîne de comparaison** : la même chaîne, rejouée avec huit outils.

Première grande partie : la chaîne principale — un outil, Qwen3-Embedding-0.6B

- **0** — **Mise en route.** Carte graphique, internet, la base réelle attachée, outils et contrôles `verifier`.
- **1** — **Lire et contrôler.** Trouver l'en-tête tout seul, nettoyer les verbatims, compter, détecter la langue, chercher les données personnelles oubliées.
- **2** — **Transformer en vecteurs.** TF-IDF puis plongements multilingues ; retirer la « signature » de chaque langue par centrage.
- **3** — **Regrouper.** HDBSCAN, fusion des groupes jumeaux d'une langue à l'autre, réglage automatique, mots et définitions de chaque problème, stabilité.
- **4** — **Repérer les incidents.** Plaintes observées contre attendues (loi de Poisson), correction des tests multiples, part de la marque dans le pic ; incident, saison ou campagne.
- **6** — **Exporter.** `resultats.js` pour le tableau de bord, contrôlé : ni verbatim ni numéro de cas.

↓

la chaîne principale est validée : ses vérifications lues, `resultats.js` écrit

↓

Deuxième grande partie : la chaîne de comparaison — huit outils

- **7** — **Comparer les outils.** La même chaîne, du centrage aux alertes, avec huit représentations, du comptage de mots à un transformer de 4 milliards de paramètres : temps, mémoire, coût, qualité. Sortie : `comparaison.js`.
- **8** — **Pour finir : ce qui peut sortir de Kaggle.** Des chiffres seulement ; aucun texte ne s'affiche ni ne sort.

Comme apprendre une recette avant d'acheter des casseroles. On réussit d'abord le plat avec une seule casserole (la chaîne principale). Ensuite seulement, on refait exactement la même recette avec huit casseroles pour voir laquelle cuit le mieux, le plus vite et pour le moins cher (la chaîne de comparaison). Comparer des casseroles sur une recette qu'on rate encore ne dirait rien.

3. Mise en route sur Kaggle

À faire AVANT la séance

Créer un compte **kaggle.com** → **Vérifier ton téléphone** (sinon : ni GPU ni internet) → Importer le notebook du TP (**File, Import**) → **Add Input** : le jeu de la base réelle

Allumer le GPU et internet

1. Panneau de droite : **Session options**
2. **Accelerator** → **GPU T4**
3. **Internet** **On**
4. Exécuter les cellules de la **partie 0**

Internet sert à télécharger les modèles (des fichiers de plusieurs centaines de mégaoctets à quelques gigaoctets), jamais à envoyer les plaintes : tous les calculs se font dans le notebook.

`[À revoir]` = la marge d'une copie corrigée : une remarque et un **indice**, au lieu d'une erreur qui arrête tout. **Lis l'indice** avant de passer à la suite.

4. Les règles de confidentialité

Le notebook tourne sur des plaintes réelles, couvertes par un accord de confidentialité que tu as signé. Les règles ci-dessous s'appliquent donc à chaque exécution.



Ne recopie aucun verbatim
ni dans un rapport, ni dans un message, ni dans une capture d'écran.

Aucun service en ligne
pour lire ou traduire les plaintes : pas de chatbot, pas de traducteur web.

Notebook privé.
Jamais « Public », jamais partagé hors de l'équipe.

Ne télécharge que l'export
(`tableau_de_bord/*.js`), jamais les vecteurs `plongements_*.npy` : on sait parfois retrouver une partie d'un texte à partir de son vecteur.

I. Parties 0 à 2 : lire, nettoyer, transformer en nombres

Avant de regrouper quoi que ce soit, on vérifie ce qu'on a reçu.

5. Partie 0 : mise en route

Trois cellules, rien à modifier

0.1 `NOM_FICHIER` : le fichier de la base réelle, déjà écrit. `AFFICHER_TEXTES` vaut `False` : aucun verbatim ne s'affiche.

0.2 installe `py3langid`, le détecteur de langue, et `babel`, pour le nom des langues en clair.

0.3 charge les outils : `verifier`, la palette des graphiques, `montrer` qui range chaque figure dans `figures/`.

Tu dois voir : `Base : RÉELLE (aucun verbatim ne sera affiché)` et le nom du fichier réel, puis `Outils prêts.`

6. Partie 1 : lire le fichier sans le connaître à l'avance

L'export de l'entreprise est un tableau Excel qui n'est pas « propre ».

	A	B	C	D	E	F
1		*lignes 1 et 2 : vides*				
2						
3	Case Number	Verbatim	Brand	Business Segment	Year Period	Blank
4	10001234	EXTERNALBonjour,j'ai acheté...	Brand 003	Chocolate	26 P05	■
5	10001302	EXTERNALHello,the bar *******...	Brand 007	Chocolate	26 P05	■
6	10001377	EXTERNALGuten Tag,die Packung...	Brand 012	Food	26 P06	■
7		*... des centaines de lignes ...*				
8		*lignes vides en fin de tableau*				
9						

l'en-tête est en ligne 3 →

à ignorer

Schéma de la disposition du fichier ; les lignes montrées sont inventées.

1.1 Un lecteur qui trouve tout seul où commencent et finissent les données

`charger_plaintes(chemin)` lit le fichier **sans en-tête et en texte**, cherche la première ligne qui contient à la fois `Case Number` et `Verbatim`, garde les cinq colonnes utiles, ignore `Blank` et les colonnes sans nom, retire les lignes vides, et rend un petit **rapport** (feuille, ligne d'en-tête, lignes vides retirées, colonnes ignorées).

Tu dois voir : l'en-tête trouvé tout seul (en **ligne 3** sur l'export), le nombre de plaintes lues, la colonne `Blank` ignorée, les cinq colonnes attendues et seulement elles.

un facteur qui cherche le nom sur l'enveloppe, au lieu de compter « troisième enveloppe en partant du haut ».

`pd.read_excel(chemin, skiprows=2, nrows=1301)` marche aujourd'hui, et **cassera en silence** demain : une ligne de titre en plus ou un fichier plus long, et on lit de travers sans aucun message.

Réflexion 1.1. Si la prochaine livraison compte 40 000 lignes, avec une ligne de titre en plus avant l'en-tête, ou arrive en CSV avec un séparateur `;`, que faut-il changer dans `charger_plaintes` ? Et dans la version `skiprows=2, nrows=1301` ?

1.2 Nettoyer les verbatims, dans le bon ordre

```
EXTERNALBonjour,j'ai acheté une tablette ******* hier.Elle avait un goût de savon
!Lien : https://exemple.org/fiche.produit
```

1. retirer `EXTERNAL` 2. retirer le lien 3. retirer les étoiles
4. une espace après la ponctuation 5. des espaces simples

```
Bonjour, j'ai acheté une tablette hier. Elle avait un goût de savon ! Lien :
```

Exemple inventé, dans la forme des verbatims de la base fictive.

Tu dois voir : plus aucun texte ne commence par `EXTERNAL`, plus d'étoiles ni de liens, les phrases sont séparées, une adresse web n'est pas coupée en morceaux.

on enlève l'étiquette collée sur la lettre avant de la lire, et on sépare les mots écrits sans espace.

Réflexion 1.2. Pourquoi retirer le lien **avant** de remettre des espaces après la ponctuation ?

1.3 Contrôler ce qu'on a reçu

Doublons de `Case Number`, verbatims vides, longueur des textes en mots, comptes par marque, segment et période.

Tu dois voir : aucun doublon, aucun verbatim vide ; la longueur médiane des plaintes, puis le nombre de marques, de segments et de périodes.

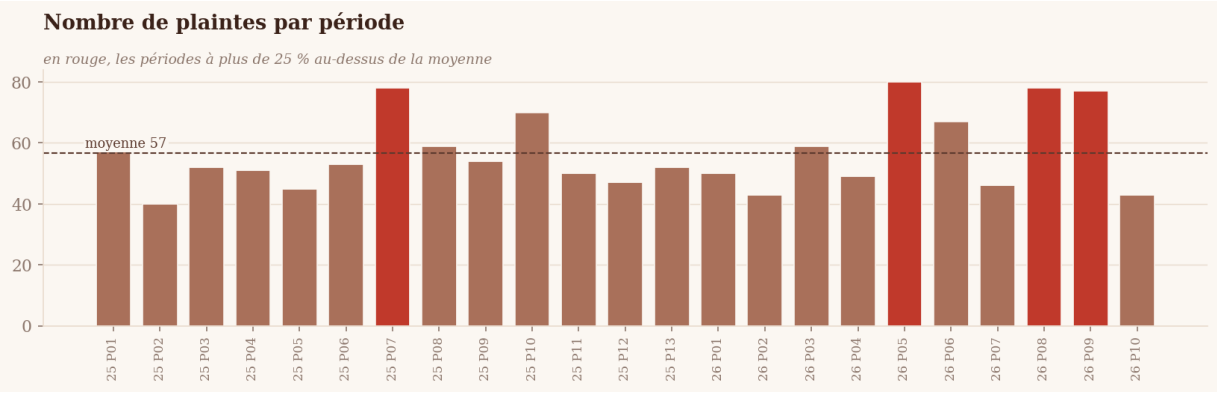


Figure 1 du notebook (base fictive) : le nombre de plaintes par période.

Comment lire ce graphique. Une barre par période (« 26 P05 » : année 2026, cinquième période). La ligne pointillée est la moyenne ; les barres rouges la dépassent de plus d'un quart. Ce graphique dit **quand** il y a eu beaucoup de plaintes, jamais **pourquoi**.

Réflexion 1.3. Le graphique montre des bosses, l'été 2025 et au printemps 2026. Est-ce un incident ?

1.4 Quelles langues ?

`py3langid.classify(texte)` rend le code de la langue et un score. La base n'a ni colonne pays ni colonne langue : la langue est le seul indice de géographie.

Tu dois voir : une langue pour chaque plainte, et **plusieurs langues** (les plaintes viennent d'Europe et du Moyen-Orient), de la plus fréquente à la plus rare, avec leur nom en clair. Quelques langues très rares sont souvent des erreurs du détecteur sur des textes courts.

la langue d'une lettre dit dans quelle boîte de tri la mettre, pas de quelle ville elle vient : l'allemand couvre plusieurs pays.

1.5 L'anonymisation a-t-elle laissé passer quelque chose ?

On **compte**, sans lire les textes, les verbatims d'origine qui contiennent encore une forme reconnaissable :

motif	expression régulière
adresse électronique	`[\w.+-]+@[\w-]+\.[\w.]+`
numéro de téléphone	`\+?\d[\d .-]{7,}\d`
suite d'au moins six chiffres	`\b\d{6,}\b`
adresse web	`https?://\S+`

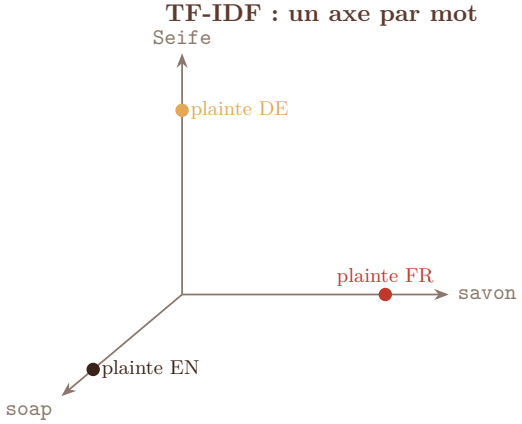
Tu dois voir : ni courriel, ni téléphone, ni longue suite de chiffres ; **40 plaintes** contiennent un lien web ; toutes citent un produit masqué par des étoiles.

Réflexion 1.5. Sur la base réelle, un compte non nul ne prouve pas une fuite, et un compte nul ne prouve pas l'absence de fuite. Pourquoi ?

7. Partie 2 : transformer chaque plainte en vecteur

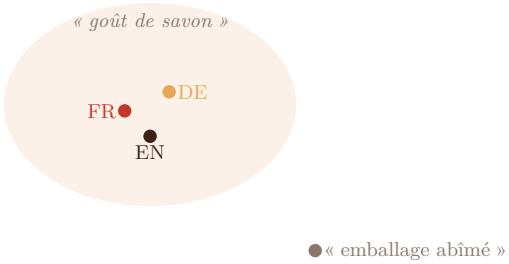
Pour regrouper des textes, il faut les transformer en nombres. Deux familles s'opposent.

TF-IDF : un axe par mot



trois mots sans rapport : les trois plaintes sont à angle droit, aussi loin que possible

plongement : un espace de sens



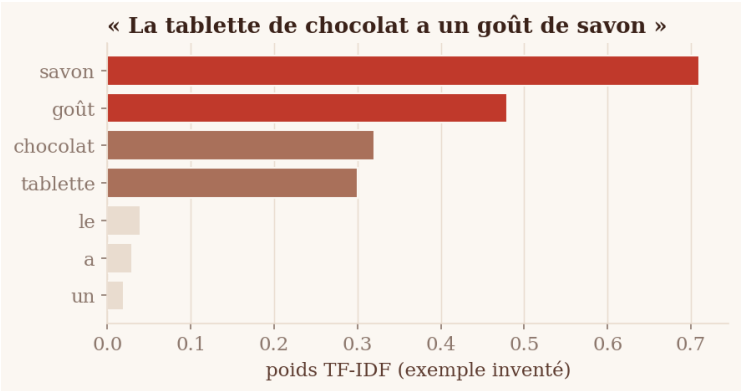
un modèle entraîné sur plus de cent langues place les trois plaintes côte à côte

2.1 TF-IDF, et à quoi ressemblent les plus proches voisins

Chaque plainte a devient un vecteur avec une coordonnée par terme t :

$$w_{t,a} = (1 + \ln \mathrm{tf}_{t,a}) \left(\ln \frac{1+n}{1+n_t} + 1 \right) \quad \text{si } \mathrm{tf}_{t,a} > 0, \qquad w_{t,a} = 0 \text{ sinon,}$$

où $\mathrm{tf}_{t,a}$ est le nombre de fois où t apparaît dans la plainte a, n le nombre de plaintes et n_t le nombre de plaintes qui contiennent t. Le vecteur est ensuite divisé par sa longueur. La proximité de deux plaintes est le **cosinus** de leurs vecteurs $\mathbf{x}_a$, $\mathbf{x}_b$: $\cos(\mathbf{x}_a, \mathbf{x}_b) = \mathbf{x}_a^\top \mathbf{x}_b / (\|\mathbf{x}_a\| \, \|\mathbf{x}_b\|)$, qui vaut 1 pour deux textes identiques et 0 pour deux textes sans mot commun.



Exemple inventé : un mot rare et répété pèse lourd, un mot partout (« le ») presque rien.

Ensuite, pour chaque plainte, on cherche sa plus proche voisine et on compte la part des plaintes dont la voisine est **dans la même langue**.

Tu dois voir : 757 mots dans le vocabulaire ; voisine dans la même langue : **100 %**.

2.2 Plongements multilingues : un transformer qui lit la plainte

« Elle avait un goût de savon ! » → jetons `Elle|avait|un|goût|de|sav|on|!` → 28 couches d'attention → un vecteur de **1 024 nombres**

Le modèle `Qwen/Qwen3-Embedding-0.6B` (Apache 2.0, plus de cent langues) reçoit chaque plainte précédée d'une consigne (« identifie le problème décrit ») et rend un vecteur de longueur 1. Dans chaque couche, chaque jeton « regarde » tous les autres pour préciser son sens : c'est l'**attention**. Si le modèle ne se charge pas, la cellule passe à `multilingual-e5-base`.

Tu dois voir : un vecteur normé par plainte ; voisine dans la même langue : **100 %**, comme le TF-IDF.

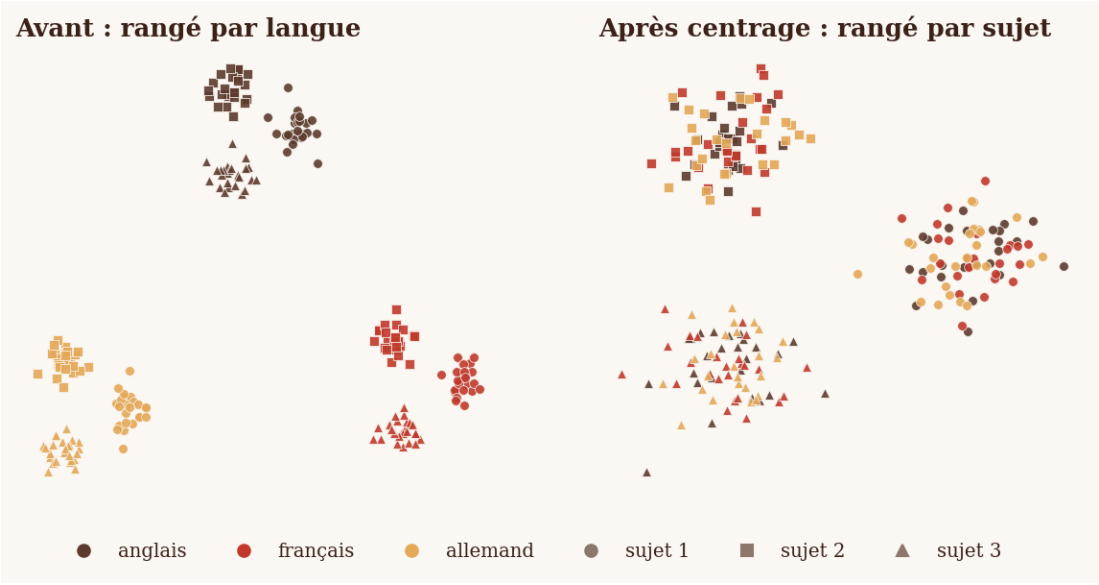
un interprète qui résume chaque lettre en une « empreinte de sens », quelle que soit sa langue.

Réflexion 2.2. Le plongement multilingue rapproche-t-il vraiment des plaintes de langues différentes ? Regarde le pourcentage.

2.3 Retirer la signature de la langue : le centrage

$$\tilde{\mathbf{e}}_i = \frac{\mathbf{e}_i - \boldsymbol{\mu}_{\ell(i)}}{\|\mathbf{e}_i - \boldsymbol{\mu}_{\ell(i)}\|},$$

où $\mathbf{e}_i$ est le plongement de la plainte i, $\ell(i)$ sa langue, et $\boldsymbol{\mu}_\ell$ la moyenne des plongements des plaintes de la langue ℓ. Une langue de moins de 10 plaintes reçoit la moyenne de toutes les plaintes.



Exemple inventé : la couleur est la langue, la forme le sujet. À gauche, on voit d'abord les langues ; à droite, après centrage, les sujets.

trois personnes qui parlent fort, doucement et avec un accent : on retire à chacune sa façon de parler, et il ne reste que ce qu'elle dit.

Tu dois voir : un vecteur centré et normé par plainte ; voisine dans la même langue : **98 %**, un peu moins qu'avant.

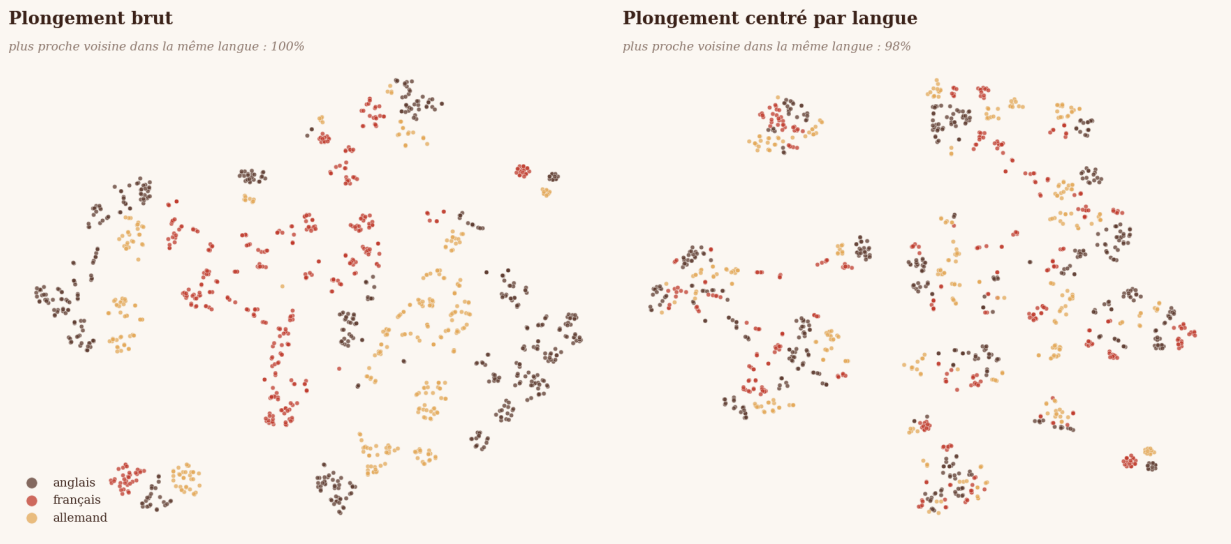


Figure 2 du notebook (base fictive) : les plaintes en deux dimensions (t-SNE), avant et après centrage.

Comment lire ce graphique. Chaque point est une plainte, la couleur est sa langue. Le t-SNE place près l'une de l'autre deux plaintes proches dans l'espace des vecteurs ; **les axes n'ont pas de sens**, seules les distances en ont. À gauche, les couleurs forment des îlots séparés. À droite, si le centrage marche, les trois couleurs se mélangent dans les mêmes paquets : chaque paquet est un sujet écrit dans plusieurs langues.

Réflexion 2.3. Le centrage retire-t-il seulement la langue ?

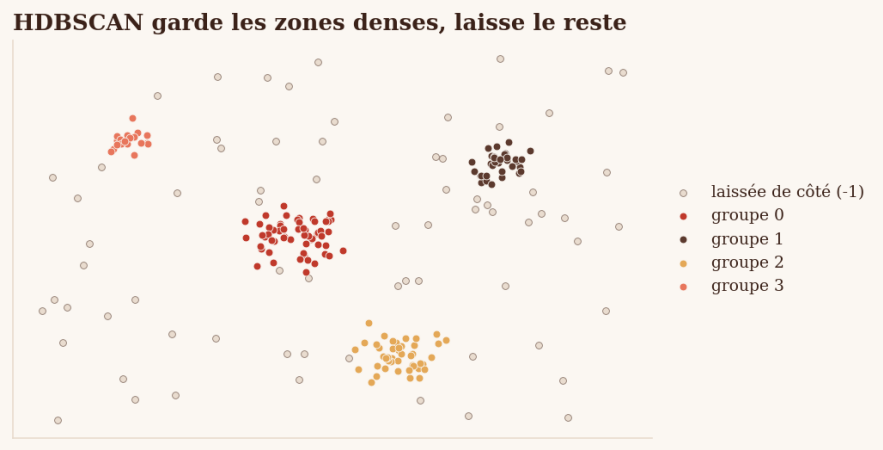
I. Partie 3 : regrouper les plaintes

Des piles par sujet, toutes langues confondues, avec un nom et une définition.

8. Partie 3 : regrouper

3.1 HDBSCAN : chercher les zones denses

On réduit d'abord les vecteurs centrés à 20 dimensions par une **analyse en composantes principales** (ACP), qui garde les 20 directions où les plaintes diffèrent le plus. Puis `HDBSCAN` trouve **lui-même** le nombre de groupes, et laisse de côté (étiquette -1) les plaintes isolées.



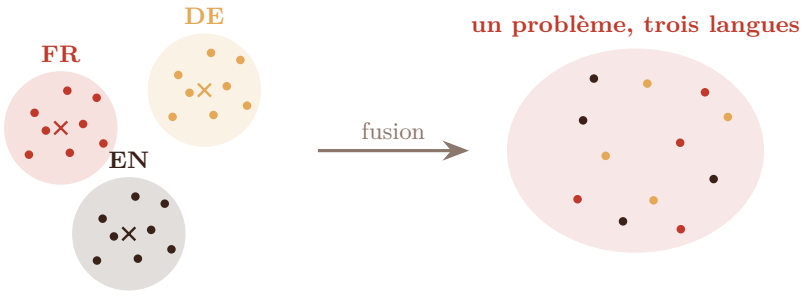
Exemple inventé : quatre zones denses, et des points isolés laissés de côté.

Tu dois voir : au moins cinq groupes, et moins de la moitié des plaintes laissées de côté.

des îles dans la mer : on ne compte que les îles, pas les rochers isolés.

3.2 Fusionner les groupes jumeaux d'une langue à l'autre

Même après centrage, HDBSCAN trouve parfois « goût de savon » en français, puis le même en allemand, juste à côté. On compare les **centres** des groupes : deux groupes dont les centres ont un cosinus supérieur à 0,4 parlent du même problème, et on les fusionne (classification ascendante hiérarchique, lien moyen, distance cosinus $1 - \cos$, seuil 0,6).



trois groupes jumeaux, centres proches (cosinus $>$ 0,4)

Tu dois voir : moins de problèmes que de groupes, et des problèmes qui réunissent **plusieurs langues**.

trois piles « factures », « Rechnungen », « bills » sur trois bureaux voisins : on les met sur une seule étagère.

3.3 Régler le regroupement sur les données elles-mêmes

Les réglages de 3.1 et 3.2 ont été choisis sur une base d'entraînement. Sur d'autres plaintes, ils peuvent tout réunir en un ou deux paquets géants. On essaie donc **18 réglages** et on choisit sans étiquette.

3 tailles minimales × 2 méthodes × 3 seuils de fusion = 18 essais

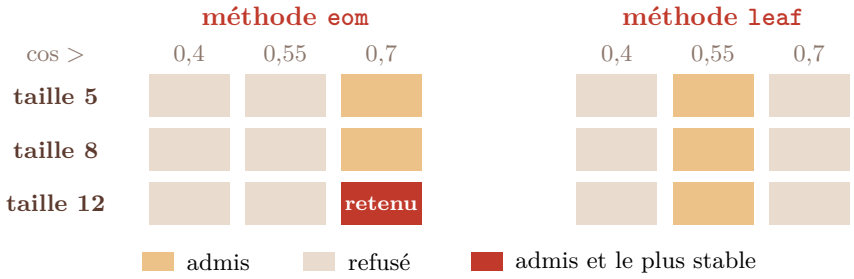


Schéma : la case « retenu » montre un réglage choisi à titre d'exemple ; les autres couleurs sont illustratives, car les essais admis dépendent des données.

Pas de géant

le plus gros problème, **campagnes exclues**, contient au plus 30 % des plaintes

Pas trop de côté

au plus la moitié des plaintes laissées de côté, hors quasi doublons

Assez de problèmes

au moins 5 problèmes

Parmi les réglages admis, on garde le **plus stable** (voir 3.5). Une **campagne** a le droit d'être grosse : on repère d'abord les **quasi doublons**, les plaintes dont la plus proche voisine TF-IDF a un cosinus supérieur à 0,9. **Plan B.** Si aucun réglage de HDBSCAN n'est admis, les plaintes ne forment pas de zones denses nettes : on passe à une classification de **Ward**, qui range chaque plainte dans l'un de k groupes ($k = 8, 12, 16, 20, 30$), avec les mêmes règles.

Tu dois voir : au moins dix-huit réglages essayés, un réglage admis trouvé, la part de quasi doublons, puis le tableau des essais et les `réglages retenus`.

3.4 Nommer chaque problème par ses mots caractéristiques

On colle les plaintes d'un problème en un seul document et on pondère chaque mot par sa fréquence dans le problème et sa rareté dans les autres (**c-TF-IDF**) :

$$W_{w,g} = \frac{c_{w,g}}{\sum_{w'} c_{w',g}} \times \ln\left(1 + \frac{A}{f_w}\right),$$

où $c_{w,g}$ est le nombre d'occurrences du mot w dans les plaintes du problème g, $f_w = \sum_g c_{w,g}$ son nombre total d'occurrences, et A le nombre moyen de mots par problème. Les mots vides (le, the, der, bonjour…) sont retirés avant.

Tu dois voir : un nom pour chaque problème, et un tableau : taille, langues, marque la

plus fréquente et sa part, mots.

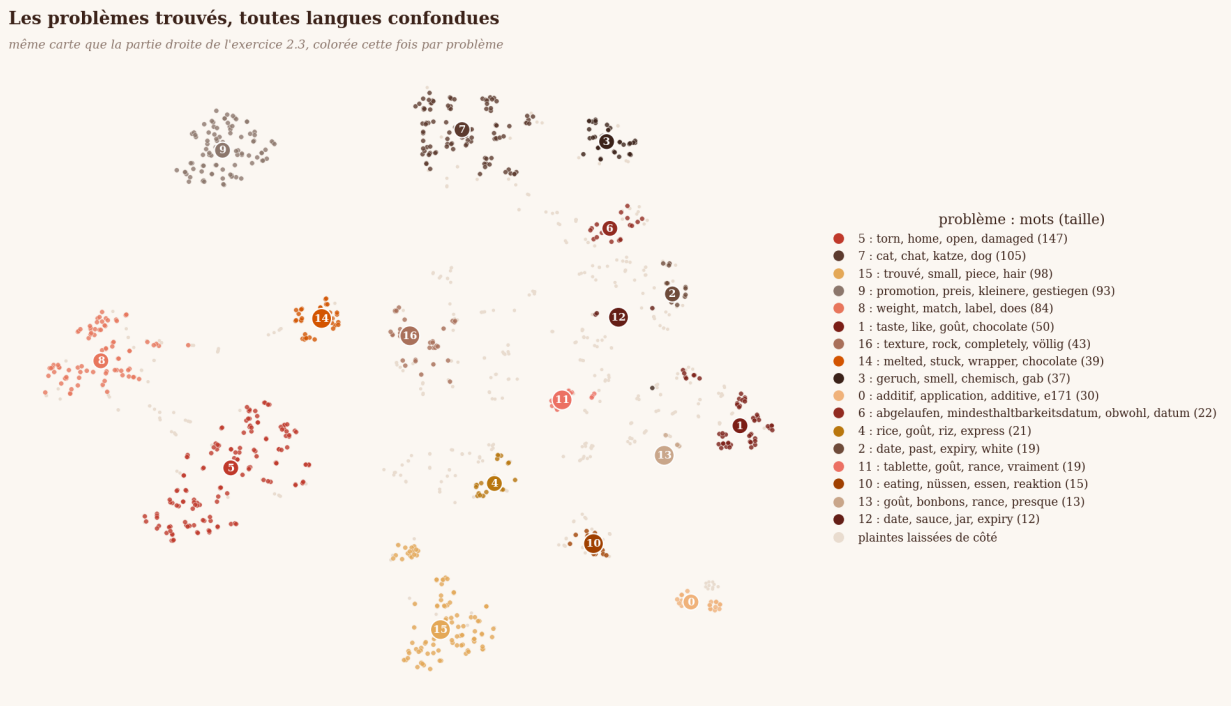
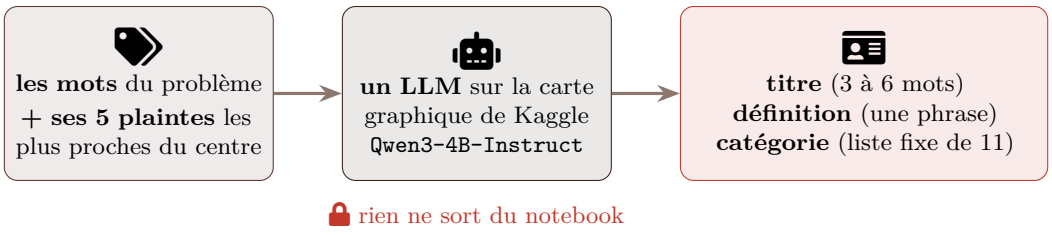


Figure 3 du notebook (base fictive) : la carte centrée, colorée par problème.

Comment lire ce graphique. C'est la carte de droite de la figure 2, colorée cette fois par **problème**. Chaque pastille numérotée est au centre d'un problème ; la légende donne ses mots les plus caractéristiques et sa taille. Les points pâles sont les plaintes laissées de côté. Un bon regroupement donne des paquets compacts d'une seule couleur ; un problème éparpillé sur toute la carte est un fourre-tout, à regarder de près.

3.4 bis Une définition pour chaque problème, écrite par un modèle de langue local



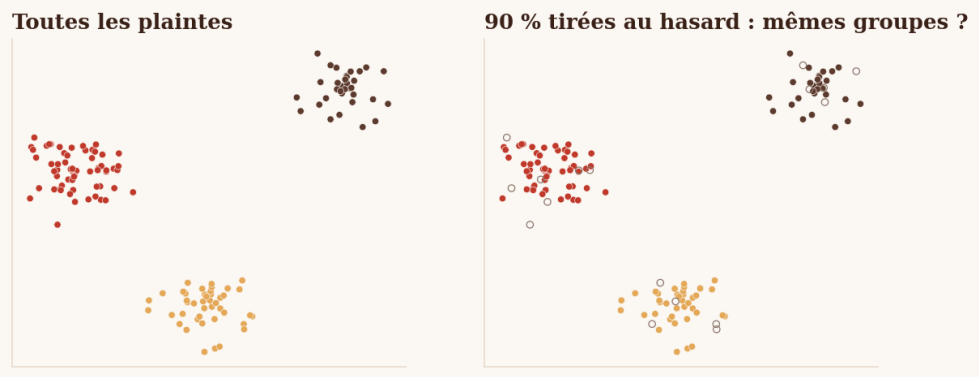
Les 11 catégories : goût ou odeur, texture ou aspect, corps étranger, emballage ou quantité, santé ou allergie, date ou fraîcheur, composition ou ingrédients, prix ou promotion, service ou livraison, compliment, autre.

Tu dois voir : une définition pour chaque problème, la plupart des titres écrits par le modèle (au moins 80 %), une catégorie de la liste pour chaque problème.

un stagiaire qui lit cinq lettres d'une pile et écrit une étiquette sur la pile : utile, mais on relit l'étiquette.

3.5 Les problèmes sont-ils stables ?

On tire trois fois 90 % des plaintes au hasard, on refait tout le regroupement, et on mesure l'accord avec le regroupement complet par l'**indice de Rand ajusté** (ARI).



Exemple inventé : retirer 10 % des plaintes ne doit pas changer les groupes.

Pour deux regroupements des mêmes n plaintes, n_{ij} est le nombre de plaintes dans le groupe i du premier et le groupe j du second, $a_i = \sum_j n_{ij}$, $b_j = \sum_i n_{ij}$, $\Sigma_a = \sum_i \binom{a_i}{2}$ et $\Sigma_b = \sum_j \binom{b_j}{2}$:

$$\mathrm{ARI} = \frac{\sum_{ij} \binom{n_{ij}}{2} - \Sigma_a \Sigma_b / \binom{n}{2}}{\frac{1}{2}(\Sigma_a + \Sigma_b) - \Sigma_a \Sigma_b / \binom{n}{2}}.$$

Il vaut 1 pour deux regroupements identiques et 0 en moyenne pour deux regroupements dus au hasard.

Tu dois voir : trois mesures d'accord, toutes au-dessus de 0,5.

I. Partie 4 : repérer les incidents

Plus de plaintes qu'attendu, sur une seule marque : c'est la signature d'un incident.

9. Partie 4 : repérer les incidents

9.1 Observé contre attendu

Prenons un problème g (par exemple « goût de savon ») et une case : une marque b et une période t. Si ce problème se répartissait au hasard entre les cases, on en attendrait

$$\lambda_{g,b,t} = \frac{n_g \times n_{b,t}}{N},$$

où n_g est le nombre de plaintes du problème g, $n_{b,t}$ le nombre de plaintes (tous problèmes confondus) de la marque b à la période t, et N le nombre total de plaintes. Le notebook note cette valeur `attendu` ; on la compare au nombre observé $o_{g,b,t}$.

problème « goût de savon » : plaintes observées (attendues)

	26 P03	26 P04	26 P05	26 P06	26 P07
Brand 001	1 (0,9)	0 (0,8)	1 (1,0)	0 (0,9)	1 (0,8)
Brand 002	0 (0,6)	1 (0,7)	0 (0,7)	1 (0,6)	0 (0,6)
Brand 003	1 (0,8)	0 (0,9)	**9 (2,0)**	1 (0,9)	0 (0,8)
Brand 004	1 (1,0)	1 (0,9)	1 (1,1)	0 (0,9)	1 (1,0)

9 plaintes là où le hasard en donnerait environ 2

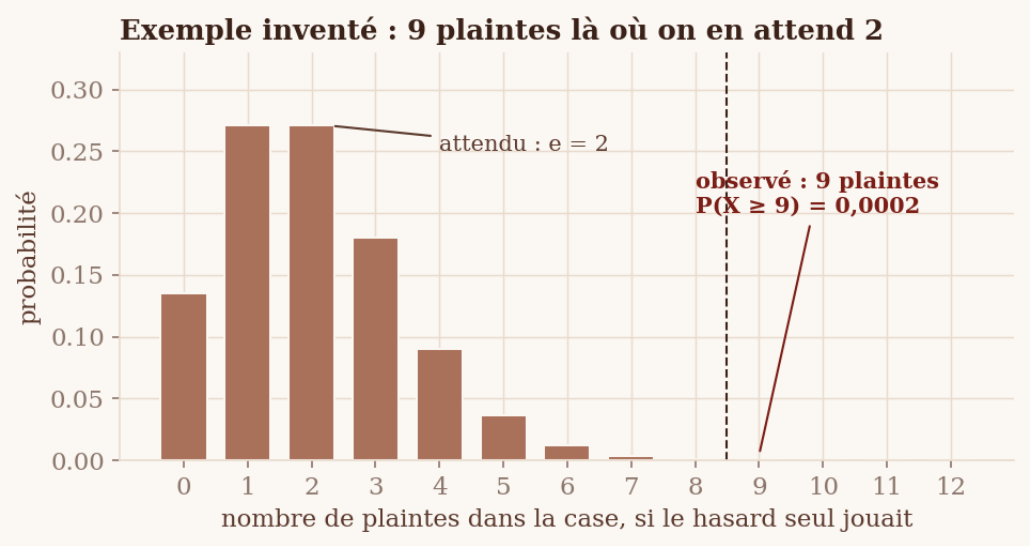
Tableau inventé, pour illustrer le calcul.

9.2 La loi de Poisson : est-ce le hasard ?

Si le hasard seul jouait, le nombre de plaintes X de la case suivrait une loi de Poisson de paramètre $\lambda = \lambda_{g,b,t}$. La **p-valeur** est la probabilité d'observer au moins autant :

$$p = P(X \geq o) = 1 - \sum_{k=0}^{o-1} \frac{\lambda^k \, \mathrm{e}^{-\lambda}}{k!},$$

où $o = o_{g,b,t}$ est le nombre observé, k parcourt les valeurs entières de 0 à $o-1$, et $\mathrm{e} \approx 2{,}718$ est la base du logarithme népérien.



Exemple inventé : avec 2 plaintes attendues, en observer 9 arrive environ 2 fois sur 10 000.

9.3 Tester des centaines de cases : la correction de Benjamini-Hochberg

Lancer cent pièces. Si on lance cent pièces dix fois, l'une d'elles finira bien par faire « pile » huit fois : ce n'est pas une pièce truquée, c'est qu'on a regardé beaucoup de pièces. Tester des centaines de cases, c'est pareil.

On range les m p-valeurs par ordre croissant, $p_{(1)} \leq \ldots \leq p_{(m)}$, puis

$$q_{(i)} = \min_{j \geq i} \min\left(1,\ \frac{m\, p_{(j)}}{j}\right).$$

Garder les cases avec $q < 0{,}01$ limite à environ 1 % la part de fausses découvertes parmi elles.

9.4 Une marque, ou tout le monde ? La part dans le pic

$$\pi_{g,b,t} = \frac{o_{g,b,t}}{o_{g,t}},$$

où $o_{g,t}$ est le nombre de plaintes du problème g à la période t, toutes marques confondues. Un incident de production touche une marque : π est grand. Une saison touche tout le monde : π est petit.

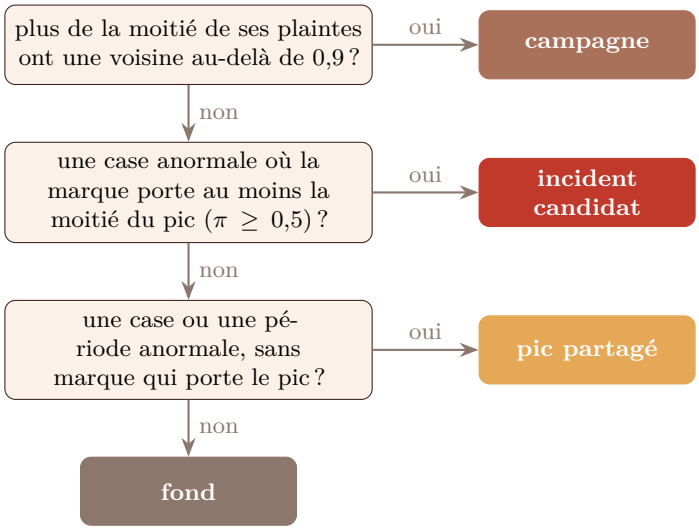
4.1 Les cases anormales

`cases_anormales(cles, seuil=0.01, o_min=5)` calcule, pour chaque problème et chaque case : observé, attendu, p-valeur, q. Elle garde les cases avec $q < 0{,}01$ et au moins 5 plaintes. On l'applique à (marque, période) en ajoutant `part_pic`, puis à la période seule.

Tu dois voir : des cases anormales sur marque et période ; toutes les valeurs q et toutes les parts entre 0 et 1.

Le sac de bonbons. Un sac où 10 % des bonbons sont rouges : en en tirant 20, on attend environ **2 rouges**, c'est λ. On en tire **9** : la **p-valeur** est la chance d'en avoir au moins autant par pure malchance ; si elle est minuscule, le sac a un problème. Mais on regarde des centaines de sacs, et l'un d'eux finira bien par sortir beaucoup de rouges par hasard : q est la p-valeur rendue plus sévère pour en tenir compte. On ne lève une alerte que si $q < 0{,}01$.

4.2 Incident, pic partagé ou campagne ? Dans cet ordre



Les cases des incidents candidats forment les `alertes`.

Tu dois voir : chaque problème a un statut : incident candidat, campagne, pic partagé ou fond. Une base peut très bien n'avoir aucun incident.

À la cantine de l'école. Les classes sont les marques, les semaines sont les périodes.

Campagne

30 élèves recopient la même lettre : « on veut des frites ». C'est une pétition, pas un problème de cuisine.

Incident candidat

Mardi, 8 élèves du CM1, et presque personne d'autre, ont mal au ventre après le yaourt. Un mauvais lot ?

Pic partagé

En décembre, toutes les classes toussent en même temps. Ce n'est pas la cantine : c'est l'hiver.

Fond

Chaque semaine, un ou deux élèves trouvent la soupe trop chaude. Rien d'anormal : c'est la vie normale.

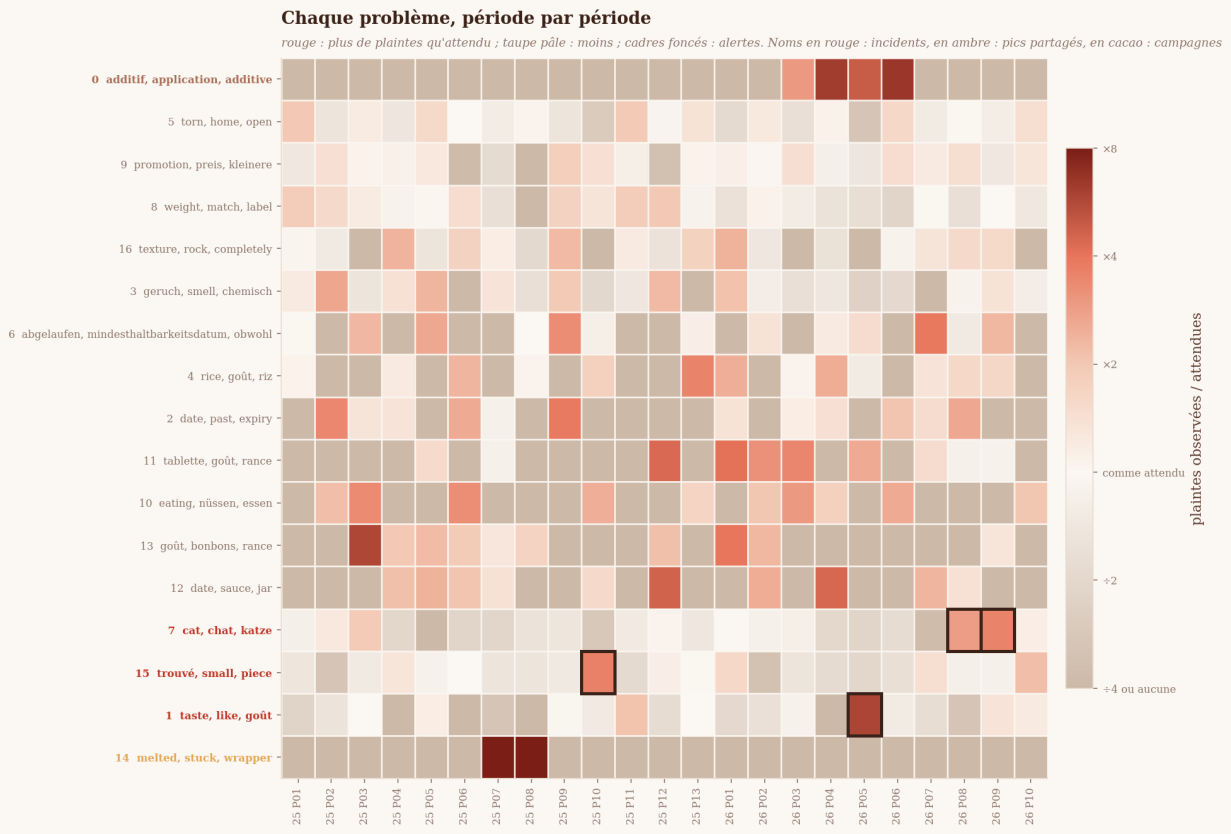


Figure 4 du notebook (base fictive) : chaque problème, période par période.

Comment lire la carte de chaleur. Une ligne par problème, une colonne par période. La couleur compare les plaintes observées à celles attendues si le problème était réparti au hasard dans le temps : crème quand c'est comme attendu, rouge de plus en plus foncé pour deux, quatre, huit fois plus, taupe pâle pour moins ou aucune. Une ligne rouge sur une ou deux cases seulement : un problème qui surgit puis disparaît. Les cases encadrées sont les **alertes** ; la couleur du nom de la ligne donne le statut (rouge : incident, ambre : pic partagé, cacao : campagne).

I. Partie 6 : exporter, et le tableau de bord

Des chiffres qui sortent, des textes qui restent : la fin de la chaîne principale.

10. Partie 6 : exporter les résultats pour le tableau de bord

`resultats.js`
comptes par période, langue, segment, marque
problèmes : taille, statut, langues
cases anormales et alertes
une position 3D par plainte (langue, période, problème)
les problèmes sont numérotés sur la base réelle

`comparaison.js`
pour chaque modèle : temps, mémoire, paramètres, nombre de problèmes, stabilité, alertes
(deuxième grande partie)

ne sortent jamais
les verbatims
les numéros de cas
les vecteurs `plongements_*.npy`
et sur la base réelle, `noms_problemes.js` reste pour la seule personne habilitée

6 Une écriture fermée par défaut

Avant d'écrire, `ecrire_export` arrondit tous les nombres à 4 décimales, puis cherche chaque numéro de cas et chaque début de verbatim dans le texte de l'export. **S'il en trouve un, le fichier n'est pas écrit** : la cellule affiche seulement le nom des champs en cause, jamais leur valeur.

Tu dois voir : aucun numéro de cas ni verbatim dans l'export, le fichier du tableau de bord écrit, une position 3D par plainte.

un portique d'aéroport : la valise ne passe que si rien d'interdit n'est dedans, et l'agent dit « poche de gauche » sans vider la valise devant tout le monde.

Pourquoi arrondir ? Un nombre écrit avec 17 chiffres, comme `0.07199999690055847`, finit par contenir par hasard une suite de 7 chiffres identique à un numéro de cas. Arrondi à `0.072`, il ne le peut plus.

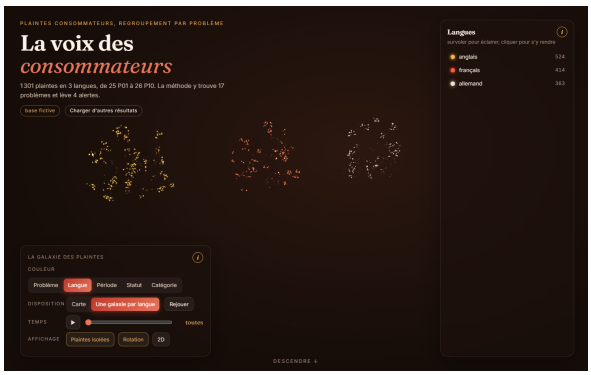
11. Le tableau de bord

Une page HTML qui fonctionne **sans internet** : on pose `resultats.js` et `comparaison.js` à côté de `tableau_de_bord.html` et on l'ouvre. Chaque graphique a un bouton *i* qui l'explique simplement.

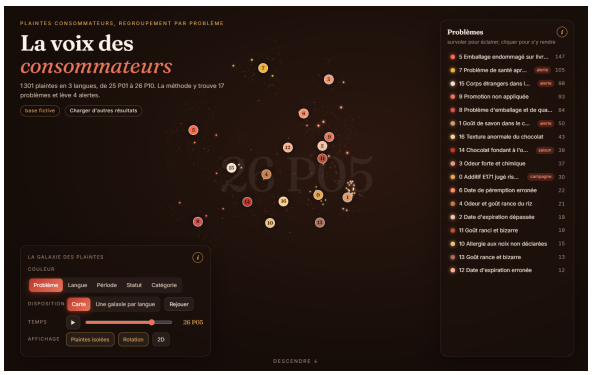


La galaxie des plaintes (base fictive) : chaque point lumineux est une plainte, chaque pastille un problème.

Comment lire la galaxie. Deux points proches racontent à peu près la même chose, même dans deux langues différentes. Un amas d'une même couleur est un problème trouvé sans qu'on dise à la méthode quoi chercher. On fait tourner à la souris ; la liste de droite donne le numéro, les mots, et un badge « alerte », « saison » ou « campagne ».



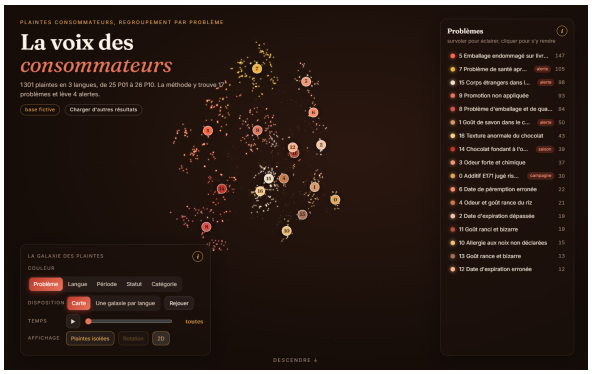
Une galaxie par langue : les mêmes amas reviennent dans chaque langue, donc les problèmes traversent les langues.



Le temps : le bouton ▶ n'allume que les plaintes d'une période ; un amas qui s'allume d'un coup est un problème qui surgit.



Un clic sur un problème : la caméra vole jusqu'à lui et sa fiche s'affiche (langues, marque la plus touchée, statut).



Le bouton 2D aplatit la galaxie pour la regarder de face.



Le mode Catégorie (base fictive) : chaque plainte prend la couleur de la famille de son problème, proposée par le modèle de langue local.



La fiche d'un problème (base fictive) : titre, définition et catégorie proposés par le modèle de langue local, puis mots, langues, marque.



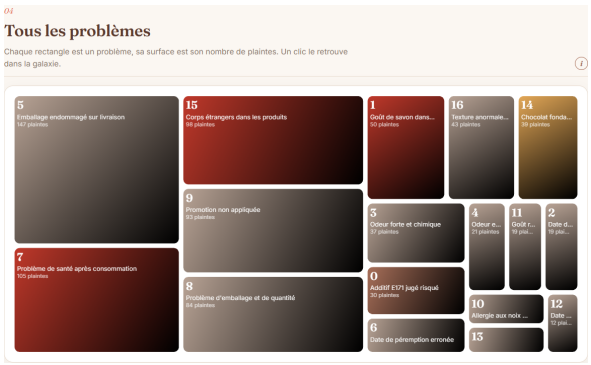
Le relief des excès : chaque tour est une case problème × période.

Comment lire le relief. La hauteur et la couleur disent combien de plaintes il y a eu par rapport à ce qu'on attendait. Un problème normal a des tours plates ; une ou deux tours qui jaillissent sur une ligne signalent un incident, une saison ou une campagne. Les tours rouge vif sont les alertes. Le bouton « À plat » montre la même chose en carte de chaleur.

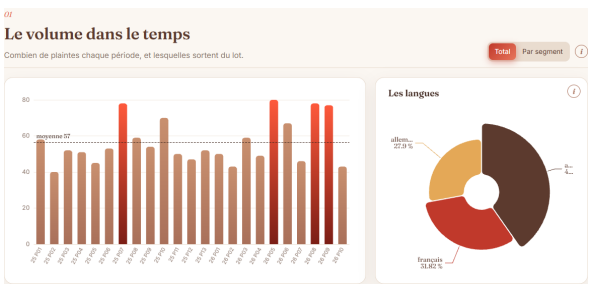


Les cartes d'alerte, puis ce qui n'est pas un incident (saison, campagne).

Comment lire une carte d'alerte. Le grand nombre : les plaintes de cette marque, pour ce problème, à cette période ; à côté, combien on en attendait, et le rapport (« ×16 » : seize fois plus). La phrase dit quelle part du pic porte la marque et quelle est la chance que ce soit le hasard. Les petites barres reprennent la figure 5 du notebook.



La mosaïque : un rectangle par problème, sa surface est son nombre de plaintes, sa couleur son statut. Un clic le retrouve dans la galaxie.



Le volume par période, total ou par segment, avec les langues à côté.



Charger de nouveaux résultats : glisser les fichiers sur la page, ou les copier à côté d'elle.



Le bouton i : chaque graphique expliqué en quelques lignes.

II. La chaîne de comparaison

La chaîne principale marche : on la rejoue avec huit outils, et on compare.

12. Avant de comparer : la chaîne principale est-elle validée ?

7.0 Le bilan de la chaîne principale

La cellule 7.0 compte les vérifications `[OK]` de la chaîne principale, liste celles `[À revoir]` et contrôle que `resultats.js` est écrit. **Lis chaque ligne `[À revoir]`** : ce n'est pas forcément une erreur (une base sans incident ne lève aucune alerte), mais tu dois savoir l'expliquer.

Tu dois voir : le nombre de vérifications réussies, la liste de celles à revoir, et `[OK]` pour « la chaîne principale a écrit resultats.js ».

on goûte le plat avant d'acheter d'autres casseroles.

Ne lance pas la comparaison sur une chaîne qui ne marche pas. Si `resultats.js` n'est pas écrit, reviens aux parties 0 à 6 : comparer huit outils sur une chaîne cassée ne dit rien de ces outils.

13. Partie 7 : comparer les outils

La chaîne principale reste la même ; seule change la façon de transformer une plainte en vecteur. Huit représentations, du comptage de mots à un transformer de 4 milliards de paramètres. Aucune n'écrit de texte, et toutes tournent dans le notebook.

représentation	origine	type	paramètres	licence
TF-IDF + SVD (100 dim.)	aucun créateur	comptage de mots	aucun	
MiniLM-L12 multilingue	SBERT, Microsoft	transformer	118 M	Apache 2.0
e5-small multilingue	Microsoft	transformer	118 M	MIT
e5-base multilingue	Microsoft	transformer	278 M	MIT
e5-large multilingue	Microsoft	transformer	560 M	MIT
BGE-M3	BAAI	transformer	568 M	MIT
Qwen3-Embedding-0.6B	Alibaba (Qwen)	dérivé d'un LLM	0,6 G	Apache 2.0
Qwen3-Embedding-4B	Alibaba (Qwen)	dérivé d'un LLM	4 G	Apache 2.0

Tableau comparatif des modèles de plongement

Les sept transformers de la partie 7 vont du modèle ultra-léger au modèle de plongement dérivé d'un LLM. Les chiffres sont ceux des versions **multilingues** utilisées dans le notebook ; les versions anglaises de MiniLM-L12 et de e5 sont plus petites (33, 110 et 335 M de paramètres).

modèle	créateur	param.	dim.	jetons max	points forts	usage idéal
MiniLM-L12 multilingue	Microsoft, adapté par SBERT	118 M	384	128	Ultra-rapide, très peu gourmand, tourne sur processeur.	Application légère, calcul en temps réel, petit budget d'infrastructure.
e5-small multilingue	Microsoft	118 M	384	512	Rapide, bon rapport précision et vitesse, avec les préfixes `query:` et `passage:`.	Prototype de RAG, moteur de recherche simple sur serveur léger.
e5-base multilingue	Microsoft	278 M	768	512	Équilibré et polyvalent, meilleure finesse sémantique.	Production standard, bon compromis entre coût et précision.
e5-large multilingue	Microsoft	560 M	1 024	512	Haute précision sémantique, capte bien les nuances.	Recherche documentaire où la précision prime sur la vitesse.
BGE-M3	BAAI (Beijing Academy of AI)	568 M	1 024	8 192	Hybride, trois en un : dense, mots-clés (creux) et multi-vecteur (ColBERT).	RAG sur documents longs qui demande à la fois le sens et les mots-clés exacts.
Qwen3-Embedding-0.6B (le modèle du TP)	Alibaba (Qwen)	0,6 G	1 024 (32 à 1 024, MRL)	32 768	Dérivé d'un LLM, sensible aux consignes, grande fenêtre de texte.	RAG moderne sur textes longs, code, multilingue exigeant.
Qwen3-Embedding-4B	Alibaba (Qwen)	4 G	2 560 (32 à 2 560, MRL)	32 768	Parmi les meilleurs du classement MTEB multilingue à sa sortie (2025).	Projet critique de très haute fidélité, avec une carte graphique dédiée.

param. : nombre de paramètres. **dim.** : taille du vecteur produit. **jetons max** : longueur maximale d'un texte, en jetons, au-delà de laquelle il est coupé. **MRL** (*Matryoshka Representation Learning*) : on peut ne garder que les premières coordonnées du vecteur et perdre peu de qualité.

Ce qu'il faut retenir pour choisir

- Ultra-vitesse, sans carte graphique : MiniLM-L12 ou e5-small.
- Un standard solide pour des textes courts ou moyens : e5-base ou e5-large.
- Retrouver des mots-clés exacts (codes, références) dans des textes longs : BGE-M3.
- Traiter de très longs documents (des PDF entiers) ou du code : Qwen3-Embedding (0.6B ou 4B).

Les plaintes du TP sont courtes et multilingues : c'est la mesure de la partie 7, pas ce tableau, qui départage les modèles sur ces données.

7.1 à 7.4 Mesurer chaque modèle, à chaîne égale

`chaine(E)` reprend centrage, réglage automatique (3.3), regroupement, fusion et alertes, sans graphiques. Pour chaque modèle on mesure :

le temps : t^{ch}, chargement du modèle (téléchargement compris la première fois), et t^{calc}, calcul des N vecteurs ; d'où $\tau = 1\,000\,t^{\text{calc}}/N$, les secondes pour 1 000 plaintes.

la mémoire : le maximum occupé sur la carte graphique.

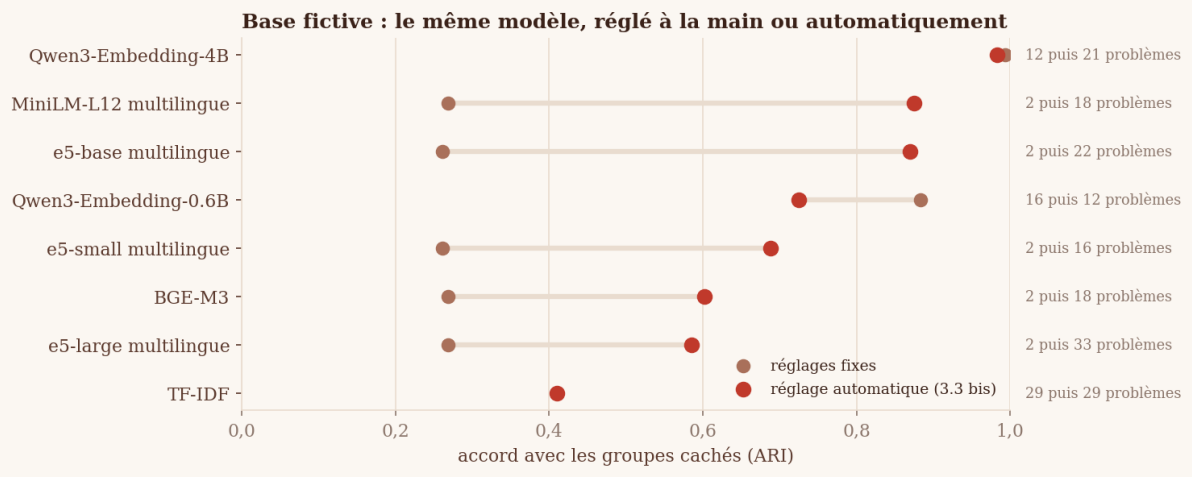
le coût pour N' plaintes, avec un tarif horaire p de la carte :

$$c(N') = \frac{t^{\text{ch}} + \tau\, N'/1\,000}{3\,600} \times p.$$

Kaggle est gratuit : c est ce que coûterait la même carte louée. On suppose que le calcul croît comme le nombre de plaintes.

la qualité : la stabilité (ARI entre trois tirages de 90 % des plaintes), le nombre de problèmes et les alertes.

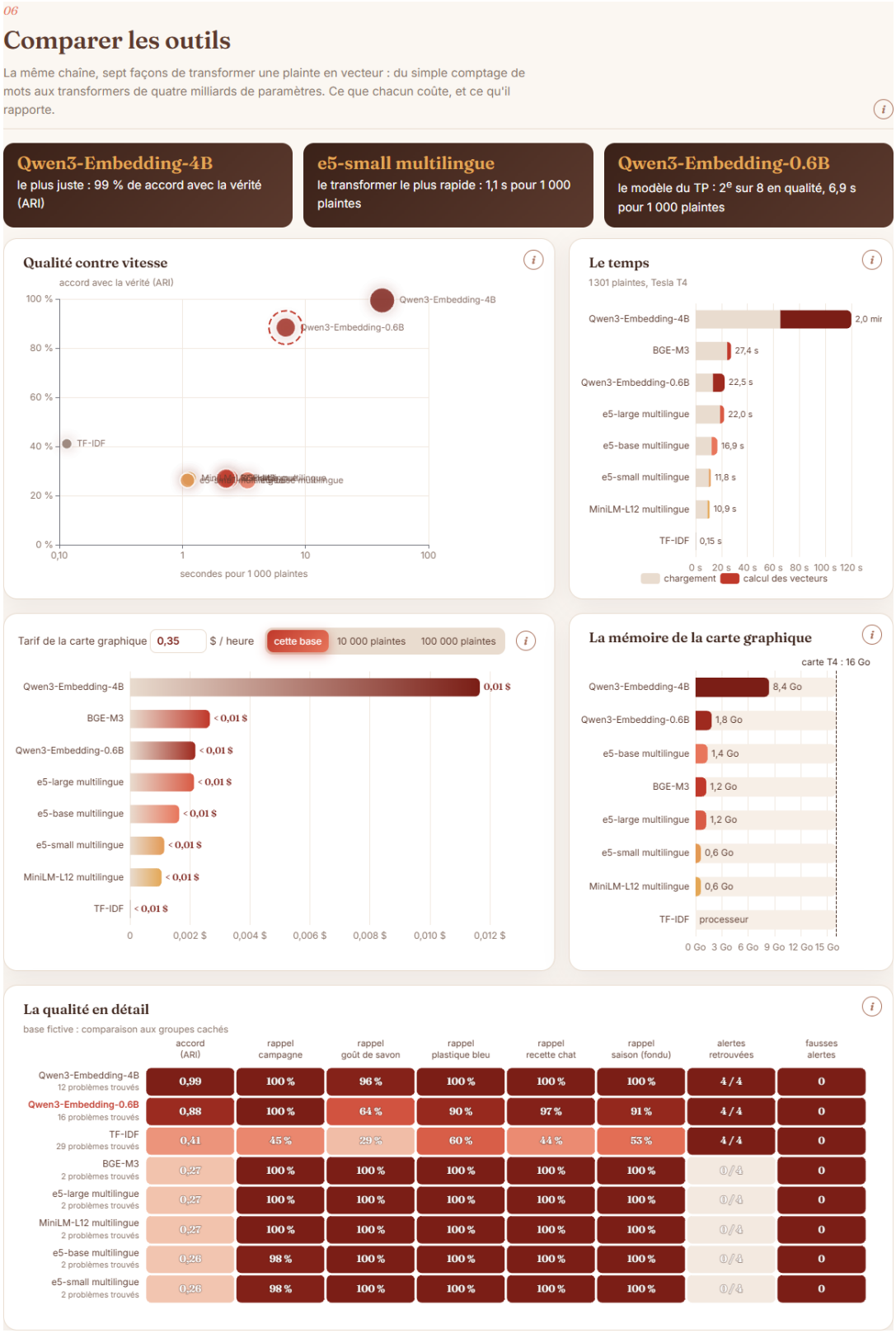
Tu dois voir : au moins cinq représentations mesurées, le TF-IDF compris, des temps positifs ; le fichier `comparaison.js` écrit, sans numéro de cas ni verbatim.



Base fictive, deux exécutions : avec les réglages fixes de 3.1 et 3.2, puis avec le réglage automatique de 3.3.

Comment lire ce graphique. Une ligne par modèle. Le point cacao est l'accord avec les groupes cachés quand tous les modèles ont les réglages choisis pour le modèle du TP ; le point rouge, quand chaque modèle reçoit son propre réglage automatique. À droite, le nombre de problèmes trouvés dans les deux cas. Plus le point est à droite, plus les problèmes trouvés coïncident avec les groupes cachés.

À discuter en équipe. Avec les réglages fixes, cinq modèles réunissent presque tout en 2 problèmes. Que vaut alors leur rappel, et pourquoi ce chiffre est-il trompeur ? Avec le réglage automatique, certains modèles gagnent beaucoup, et le modèle du TP perd un peu : que mesure exactement la règle de choix, et qu'est-ce qu'elle ne mesure pas ?



Rubrique 06 du tableau de bord sur la base fictive, première exécution (réglages fixes).

Comment lire la rubrique 06. Le podium donne le modèle le plus juste, le transformer le plus rapide et la place du modèle du TP. Dans « Qualité contre vitesse », chaque bulle est un modèle (taille : son nombre de paramètres) ; les bons choix sont en haut à gauche.

Le temps sépare chargement et calcul ; le coût se recalcule quand on change le tarif ou le nombre de plaintes ; la mémoire montre la limite de 16 Go d'une carte T4.

14. Partie 8 : ce qui peut sortir de Kaggle

1. **Add Input** : le jeu de données **privé** de la base réelle
2. cellule 0.1 : le nom du fichier est déjà écrit, rien à modifier
3. tout exécuter : aucun verbatim ne s'affiche

Ce qui peut sortir de Kaggle

- ✔ le rapport de lecture, les comptes par langue, marque, période
- ✔ le nombre et la taille des groupes, la stabilité
- ✔ les cases anormales (numéro du problème, marque, période, observé, attendu, q)
- ✔ `resultats.js` et `comparaison.js`

Ce qui ne sort pas

- ✖ les verbatims et les numéros de cas
- ✖ les vecteurs `plongements_*.npy`
- ✖ les mots et définitions des problèmes, sauf accord écrit de l'entreprise
- ✖ les figures 3 à 5, dont les légendes sont faites de ces mots

Sur de vraies plaintes, des réglages trouvés ailleurs ne tiennent pas forcément. Les vraies lettres sont plus longues, plus variées, et peuvent contenir une grosse campagne. C'est exactement pour cela que l'exercice 3.3 choisit le réglage sur les données elles-mêmes, avec un plan B.

Lexique et pistes

Les mots du TP, en une phrase chacun ; puis ce qu'on pourrait faire ensuite.

15. Lexique

mot	en une phrase
verbatim	le texte libre écrit par le consommateur, tel qu'il est arrivé.
apprentissage non supervisé	on cherche une structure dans des données sans étiquette : ici, des groupes de plaintes.
TF-IDF	une coordonnée par mot ; un mot pèse lourd s'il est fréquent dans la plainte et rare ailleurs.
cosinus	la proximité de deux vecteurs : 1 s'ils pointent dans la même direction, 0 s'ils n'ont rien en commun.
jeton	le morceau de texte que lit un transformer : un mot fréquent, ou un bout de mot rare.
transformer	un réseau de neurones où chaque jeton « regarde » tous les autres (l'attention) pour préciser son sens.
plongement	le vecteur de sens qu'un transformer donne à un texte entier.
centrage par langue	retirer à chaque vecteur la moyenne de sa langue, pour ne garder que ce que la plainte dit.
t-SNE	une méthode qui dessine en 2 ou 3 dimensions des points proches dans un grand espace ; les axes n'ont pas de sens.
ACP	l'analyse en composantes principales : garder les quelques directions où les données varient le plus.
HDBSCAN	un regroupement par densité, qui trouve seul le nombre de groupes et laisse de côté les points isolés.
Ward	une classification hiérarchique qui range chaque point, en fusionnant les groupes qui augmentent le moins la dispersion.
groupes jumeaux	le même problème trouvé deux fois, une fois par langue.

mot	en une phrase (suite)
c-TF-IDF	le TF-IDF calculé sur un groupe entier, pour trouver ses mots les plus caractéristiques.
LLM	un grand modèle de langue qui écrit du texte ; ici, il tourne dans le notebook et propose titres et définitions.
ARI	l'indice de Rand ajusté : l'accord entre deux regroupements, 1 s'ils sont identiques, 0 au hasard.
loi de Poisson	la loi du nombre d'événements rares quand on en attend en moyenne λ.
p-valeur	la probabilité d'observer au moins autant si le hasard seul jouait.
Benjamini-Hochberg	une correction des p-valeurs quand on fait beaucoup de tests à la fois.
part dans le pic	la part des plaintes d'un problème, sur une période, qui vient d'une seule marque.
quasi doublon	une plainte dont la plus proche voisine TF-IDF a un cosinus supérieur à 0,9.
incident candidat	un problème en excès sur une marque et une période, porté surtout par cette marque.
pic partagé	un problème en excès sur une période chez plusieurs marques à la fois.
campagne	un groupe de messages presque identiques : une mobilisation, pas un défaut.
rappel, précision	la part d'un groupe caché retrouvée ensemble ; la part d'un groupe trouvé qui vient du groupe caché.

16. Pour aller plus loin

Traduire avant de comparer

Au lieu de centrer par langue, traduire toutes les plaintes dans une seule langue avec un modèle de traduction qui tourne dans le notebook. Plus lourd, mais sans le risque de retirer une partie du sens avec la langue.

La date de l'événement

Une plainte peut arriver des semaines après l'achat. Si l'entreprise fournit la date de fabrication ou le numéro de lot, l'alerte pourrait porter sur le lot plutôt que sur la période de la plainte.

D'autres cases que la marque

Quand une marque domine la base, comparer les marques n'a plus de sens : on peut tester l'excès par segment, par pays (si la colonne existe), ou par période seule.

Une évaluation humaine

Faire relire, par une personne habilitée, quelques plaintes de chaque problème et noter si le titre proposé par le LLM est juste : c'est la seule façon de mesurer la qualité sur la base réelle.

D'autres regroupements

Réduire les vecteurs avec UMAP avant HDBSCAN, ou essayer une modélisation de sujets complète (BERTopic), et les ajouter à la chaîne de comparaison, à côté des huit outils.

Une alerte au fil de l'eau

Recalculer chaque semaine les cases anormales sur les nouvelles plaintes seulement, et n'alerter que sur ce qui change : c'est ainsi qu'un tel outil servirait au quotidien.

Une question ? Lève la main, l'encadrante passe.

Annexe : les outils expliqués

babel, py3langid, les expressions régulières du nettoyage,
et le centrage par langue expliqué avec des notes de classe.

17. babel : le nom des langues en clair

Ce que c'est

`babel` est une bibliothèque Python d'**internationalisation** : pour chaque langue d'affichage, elle connaît les noms des langues et des pays, les formats de dates, de nombres et de monnaies. Elle les tire du **CLDR** (*Common Locale Data Repository*), la base de référence du consortium Unicode, celle qu'utilisent aussi les navigateurs et les systèmes d'exploitation.

Ce qu'elle fait dans le notebook

Une seule chose : écrire en clair le nom des langues détectées. Le détecteur de langue (ci-dessous) ne renvoie qu'un **code**, comme `fr`, `nl` ou `ary`. babel le traduit en nom français :

```
from babel import Locale
NOMS_CLDR = Locale("fr").languages # un dictionnaire : code -> nom en français
NOMS_CLDR["nl"] # 'néerlandais'
NOMS_CLDR["ary"] # 'arabe marocain'
```

Quelques codes rares que le CLDR ne nomme pas sont complétés à la main dans `COMPLEMENT_LANGUES` (par exemple `zxx`, « sans contenu linguistique »). Le nom affiché est donc, dans l'ordre : le complément s'il existe, sinon le nom donné par babel, sinon le code lui-même.

Installée en 0.2

Elle n'est pas dans l'image Kaggle par défaut : la cellule 0.2 l'installe avec `py3langid`. Si elle manque, rien ne plante : les langues s'affichent par leur code.

Ne touche pas aux plaintes

Elle ne lit aucun texte et n'envoie rien sur internet. Elle ne joue aucun rôle dans les vecteurs, les groupes ou les alertes : elle sert seulement à l'affichage.

18. py3langid : quelle langue pour chaque plainte

Ce que c'est

`py3langid` est un **détecteur de langue** : on lui donne un texte, il renvoie le code de la langue la plus probable. C'est la version modernisée de `langid.py`. Son modèle, un petit modèle probabiliste (Bayes naïf), a appris sur des textes de plus d'une centaine de langues quelles **suites de caractères** sont typiques de chacune : « ijk » en néerlandais, « sch » en allemand, « eau » en français. Il est livré avec le paquet et tourne sur le processeur, sans internet et sans carte graphique, en une fraction de seconde pour toute la base.

Ce qu'il fait dans le notebook (exercice 1.4)

```
import py3langid
py3langid.classify("Bonjour, le chocolat avait un goût de savon.")
# ('fr', -209.3) : le code de la langue, puis un score
plaintes["langue"] = [py3langid.classify(t)[0] if t else "?" for t in plaintes.texte]
```

`classify` renvoie deux valeurs ; on garde la première, le code (`[0]`). Le score est une **log-probabilité** : plus il est grand, plus le modèle est sûr, mais ce n'est pas un pourcentage. Un texte vide reçoit `"?"`.

Tu dois voir : une langue pour chaque plainte, et une répartition avec quelques langues principales.

La langue n'est pas le pays. L'anglais, le français et l'allemand couvrent chacun plusieurs pays. **Et le détecteur se trompe sur les textes très courts** ou pleins de chiffres et de références : une langue qui ne compte qu'une ou deux plaintes est souvent une erreur de détection, pas un vrai pays de plus.

19. Les expressions régulières du nettoyage (exercice 1.2)

19.1 re, c'est quoi ?

`re` est le module de Python pour les **expressions régulières** (*regular expressions*). Une expression régulière est un **motif** qui décrit une *forme* de texte, au lieu d'un texte exact. Au lieu de chercher le lien précis `https://exemple.io`, on décrit la forme de tous les liens : « commence par http, peut-être un s, puis ://, puis des caractères sans espace ».

19.2 Comment se lit re.sub

`re.sub` veut dire *substitute*, remplacer. Elle prend toujours trois choses, dans cet ordre :

```
re.sub( motif , remplacement , texte )
```

1. le motif	2. le remplacement	3. le texte
ce qu'on cherche	par quoi on le remplace	où on cherche

Elle renvoie le texte modifié. Exemple : `re.sub(r"\*{2,}", " ", texte)` se lit « dans `texte`, remplace chaque suite de deux étoiles ou plus par une espace ».

Le `r` devant le motif

`r"..."` est un texte « brut » : Python garde les barres \ telles quelles et les passe au motif.

`texte`, à la fin

C'est la variable qui contient la plainte à nettoyer. Le résultat de chaque ligne remplace `texte`, et la ligne suivante repart de lui.

19.3 Les briques pour lire les motifs

symbole	veut dire
`^`	le tout début du texte
`\s`	un « blanc » : espace, tabulation, retour à la ligne
`\S`	tout sauf un blanc
`*`	0 fois ou plus le symbole d'avant
`+`	1 fois ou plus le symbole d'avant
`?`	0 ou 1 fois le symbole d'avant : il est facultatif
`{2,}`	2 fois ou plus le symbole d'avant
`\*`	une vraie étoile (la barre empêche de la lire comme « 0 fois ou plus »)
`[.!?]`	un seul caractère parmi ceux de la liste
`[^\W\d_]`	une lettre, de n'importe quel alphabet (ni chiffre, ni ponctuation, ni _)
`(...)` et `\1`	on capture un morceau, puis on le remet tel quel dans le remplacement
`(?=...)`	regarde ce qui suit, sans le consommer ni le modifier

19.4 Les cinq étapes de nettoyer, une par une

```
def nettoyer(texte):
    texte = re.sub(r"^\s*EXTERNAL", "", texte) # 1
    texte = re.sub(r"https?://\S+", " ", texte) # 2
    texte = re.sub(r"\*{2,}", " ", texte) # 3
    texte = re.sub(r"([.!?;:,virgule arabe])(?=[^\W\d_])", r"\1 ", texte) # 4
    return re.sub(r"\s+", " ", texte).strip() # 5
```

#	ce que fait le motif	exemple
1	au début du texte (`^`), des blancs ou rien, puis le mot EXTERNAL : supprimé	`EXTERNALBonjour` → `Bonjour`
2	`http`, un `s` facultatif, `://`, puis tout jusqu'au premier blanc : le lien entier devient une espace	`voir https://exemple.io/aB3` → `voir ␣`
3	deux étoiles ou plus (les produits masqués) : une espace	`acheté ******* hier` → `acheté ␣␣␣hier`
4	une ponctuation suivie directement d'une lettre : on garde la ponctuation (`\1`) et on ajoute une espace	`Bonjour,J'ai` → `Bonjour, J'ai` ; `3.5` ne bouge pas
5	les espaces simples : chaque groupe de blancs devient une seule espace, puis `.strip()` enlève les blancs du début et de la fin	`acheté ␣␣␣hier` → `acheté hier`

Le signe ␣ montre une espace.

19.5 L'étape 5 et strip, en détail

5 Des espaces simples

La dernière ligne fait deux choses, de gauche à droite.

1. `re.sub(r"\s+", " ", texte)` : `\s+` veut dire « un blanc ou plus, d'affilée ». Chaque groupe de blancs, qu'il compte 1 ou 10 espaces, tabulations ou retours à la ligne, devient **une seule espace**.
2. `.strip()` n'est pas du `re` : c'est une méthode de tout texte Python. Elle enlève les blancs au **début** et à la **fin**, jamais au milieu : `"  Bonjour le monde  ".strip()` donne `"Bonjour le monde"`.

Cette étape est nécessaire parce que les étapes d'avant **créent** des espaces en trop :

```
départ : "EXTERNALBonjour, j'ai acheté ******* hier"
après 1 : "Bonjour, j'ai acheté ******* hier"
après 3 : "Bonjour, j'ai acheté hier" (trois espaces)
après 5 : "Bonjour, j'ai acheté hier" (une seule)
```

Un lien en fin de message devient lui aussi une espace finale, que `.strip()` retire.

après avoir découpé des étiquettes dans une lettre, on referme les trous pour que le texte se lise d'une traite.

20. Le centrage par langue, avec des notes de classe (exercice 2.3)

20.1 Deux professeurs, deux façons de noter

La classe du professeur dur

élève	note
Ali	6
Bea	8
Chloé	9
Dan	10
Léa	**12**

Moyenne de la classe : $(6+8+9+10+12)/5 = 45/5 = 9$.

La classe du professeur large

élève	note
Tom	**12**
Uma	14
Vic	15
Wael	16
Zoé	18

Moyenne de la classe : $(12 + 14 + 15 + 16 + 18)/5 = 75/5 = 15$.

20.2 Même note, deux histoires opposées

Léa : $12 - 9 = +3$

Léa a 3 points **de plus** que ce que son professeur donne d'habitude. C'est la **meilleure** de sa classe.

Tom : $12 - 15 = -3$

Tom a 3 points **de moins** que ce que son professeur donne d'habitude. C'est le **dernier** de sa classe.

Les deux ont 12. Le $+3$ et le -3 montrent la différence ; le 12 brut la cachait. Le **centrage**, c'est juste ça : **remplacer la note par l'écart à la moyenne de son propre professeur**. On ne compare plus des notes, on compare ce que chaque élève a de plus ou de moins que sa classe.

20.3 Le lien avec les plaintes

dans la classe	dans le notebook
le professeur	la **langue** de la plainte
la note d'un élève	le **plongement** $\mathbf{e}_i$ de la plainte : 1 024 nombres au lieu d'un seul
la moyenne de la classe	la moyenne $\boldsymbol{\mu}_\ell$ des plongements de toutes les plaintes de la langue ℓ
l'écart +3 ou −3	$\mathbf{e}_i - \boldsymbol{\mu}_{\ell(i)}$: ce que la plainte dit **de plus** que les autres plaintes de sa langue

Un plongement mélange deux choses : ce que dit la plainte (le sujet) et la façon dont sa langue s'exprime (la « façon de noter » du professeur). Deux plaintes allemandes sur des sujets différents se ressemblent un peu, simplement parce qu'elles sont en allemand. En retirant à chaque plainte la moyenne de sa langue, on retire cette part commune, et il reste le sujet : une plainte française et une plainte allemande sur le goût de savon se retrouvent alors côte à côte.

Puis on renorme

Après la soustraction, on divise par la longueur du vecteur (le dénominateur de la formule de l'exercice 2.3), pour que toutes les plaintes aient la même longueur, 1.

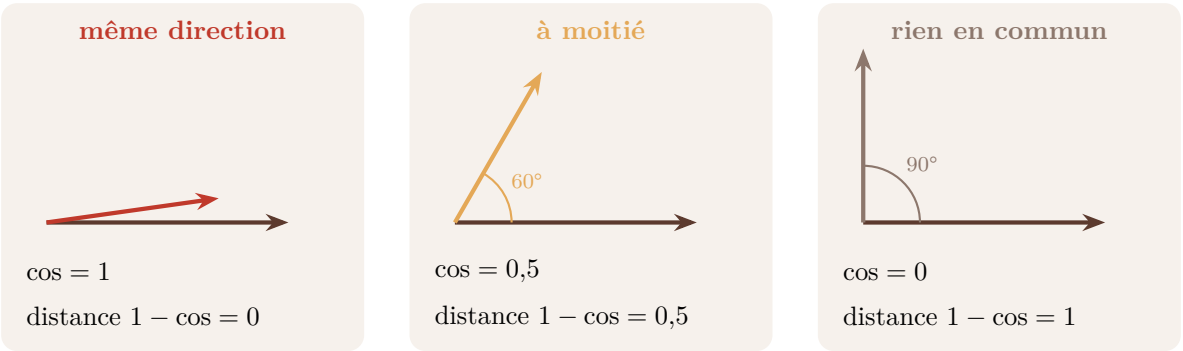
Les petites classes

Une moyenne sur deux élèves ne dit rien du professeur. De même, une langue de moins de 10 plaintes reçoit la moyenne de **toutes** les plaintes, pas la sienne.

21. Cosinus : similarité et distance (exercices 2.1 et 3.2)

21.1 Le cosinus regarde l'angle entre deux flèches

Chaque plainte, et chaque centre de groupe, est une **flèche** (un vecteur). Le **cosinus** ne regarde pas la longueur des flèches, seulement l'**angle** entre elles : il dit si elles pointent dans la même direction, donc si les deux textes parlent de la même chose.



Flèche rouge : une plainte ; flèche colorée : une autre plainte. Seul l'angle compte.

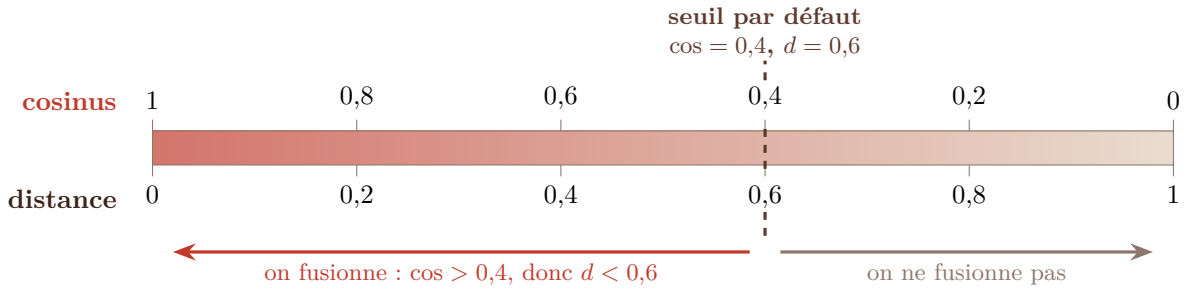
21.2 Deux façons de dire la même chose

La similarité cosinus

Grande quand les textes se ressemblent : 1 pour la même direction, 0 pour rien en commun. C'est elle qu'on lit dans les seuils du TP (« cosinus $> 0{,}4$ », voisine « au-delà de 0,9 »).

La distance cosinus

Petite quand les textes se ressemblent : $d = 1 - \cos$. C'est ce que demandent certaines fonctions de scikit-learn, comme la fusion des jumeaux (`metric="cosine"`).



Comment lire la règle. Le haut donne le cosinus, le bas la distance au même endroit : les deux nombres d'une même graduation font toujours 1 à eux deux. « Fusionner deux groupes si le cosinus de leurs centres dépasse 0,4 » et « fusionner si leur distance est inférieure à 0,6 » désignent donc exactement la même zone, à gauche du trait. Dans le code, on règle le cosinus et la conversion se fait toute seule : `distance_threshold = 1 - cosinus_min`. Le réglage automatique (exercice 3.3) peut monter le seuil à cos $> 0{,}7$, soit $d < 0{,}3$: il ne fusionne alors que des groupes très proches.

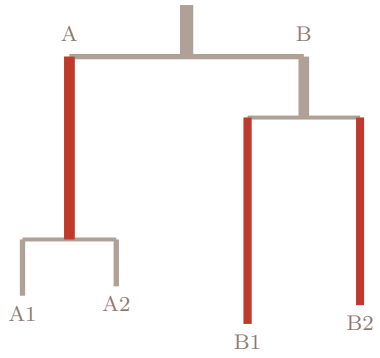
deux personnes qui montrent une direction du doigt : peu importe la longueur du bras, on regarde seulement si elles montrent le même endroit.

22. Trois façons de faire les tas : eom, leaf et Ward (exercices 3.1 et 3.3)

22.1 D'abord, l'arbre de HDBSCAN

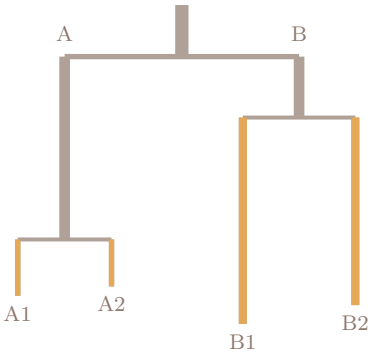
HDBSCAN regarde les plaintes comme des îles sur une carte, et fait **baisser le niveau de la mer** petit à petit. Au début, tout est une seule grande île. Quand l'eau monte entre deux zones moins denses, l'île se coupe en deux, puis encore en deux… On obtient un **arbre** : chaque branche est un groupe possible, et sa **longueur** dit combien de temps il a tenu avant de se couper. Une branche de moins de `taille_min` plaintes ne compte pas : ses plaintes deviennent du bruit (étiquette -1).
Il reste à choisir **quelles branches garder**. C'est là qu'interviennent les deux méthodes.

`eom` : les branches qui durent



3 groupes : A entier, B1, B2

`leaf` : les bouts de l'arbre



4 groupes : A1, A2, B1, B2

Arbre inventé. En couleur, les branches gardées : chaque plainte ne peut être que dans une seule.

`eom` : « excess of mass »

Garde les branches qui **durent le plus longtemps**, en tenant compte de leur taille. Si une branche dure plus que ses deux sous-branches réunies, on la garde entière (A) ; sinon on garde les sous-branches (B1 et B2). Résultat : **moins de groupes, plus gros et plus solides**. C'est le choix par défaut de HDBSCAN.

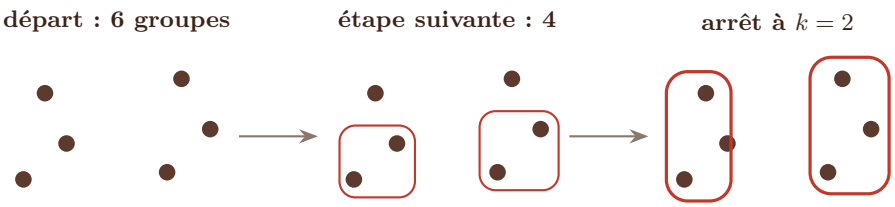
`leaf` : les feuilles

Garde tous les **bouts de l'arbre**, les plus petites branches qui ne se coupent plus. Résultat : **plus de groupes, plus petits et plus homogènes**. Utile quand on cherche des problèmes fins, au risque de couper un même problème en morceaux.

une famille qui partage un gâteau : `eom` coupe les grosses parts qui ont du sens (le côté chocolat, le côté fraise), `leaf` coupe en toutes petites parts.

22.2 Le plan B : Ward

Si aucun des 18 réglages de HDBSCAN n'est admis, c'est que les plaintes ne forment pas d'îles nettes. On change alors de méthode : la classification de **Ward** part dans l'autre sens. Au début, chaque plainte est seule dans son groupe ; à chaque étape, on **colle les deux groupes** dont la fusion mélange le moins les plaintes, et on s'arrête quand il reste k groupes.



Exemple inventé avec 6 plaintes : à chaque étape, on colle les deux groupes les plus proches au sens de Ward.

« Mélanger le moins » veut dire faire grandir le moins possible la **dispersion à l'intérieur des groupes** :

$$W = \sum_{h} \sum_{i \in h} \|\mathbf{z}_i - \bar{\mathbf{z}}_h\|^2,$$

où h parcourt les groupes, $\mathbf{z}_i$ est le vecteur de la plainte i et $\bar{\mathbf{z}}_h$ la moyenne du groupe h. Ward choisit à chaque étape la fusion qui augmente W le moins.

	HDBSCAN (`eom` ou `leaf`)	Ward (plan B)
nombre de groupes	trouvé tout seul	fixé d'avance : $k = 8, 12, 16, 20$ ou 30
plaintes isolées	laissées de côté (-1)	toujours rangées dans un groupe
ce qu'il cherche	des zones denses, de n'importe quelle forme	des groupes compacts, plutôt ronds
quand l'utiliser	des îles nettes dans les données	pas d'îles nettes : un continuum

Ward range tout, même ce qui ne ressemble à rien. Il n'y a plus de plaintes de côté : une plainte bizarre finit quand même dans un groupe. C'est pourquoi on garde les mêmes règles d'admission (pas de géant, assez de problèmes) et la fusion des jumeaux après Ward.

22.3 HDBSCAN et Ward : deux arbres, une seule différence qui compte

Les deux méthodes sont **hiérarchiques**. Toutes les deux construisent un arbre de groupes emboîtés, HDBSCAN en regardant la mer monter, Ward en collant les groupes deux à deux, puis décident où couper cet arbre. Mais **seul HDBSCAN regarde la densité**. Ward ne la regarde jamais : il regarde la **dispersion**. À chaque étape, il colle les deux groupes dont la fusion fait le moins grossir la dispersion totale W, et il s'arrête quand il reste k groupes.

HDBSCAN

Un point isolé au milieu de nulle part, HDBSCAN le **laisse de côté** (-1), parce qu'il n'est pas dans une zone dense. Il cherche **où c'est serré** et accepte de ne pas tout ranger.

Ward

Le même point isolé, Ward le **colle de force** au groupe le moins loin, parce qu'il doit tout ranger. Il **range tout** et cherche des paquets ronds et équilibrés.

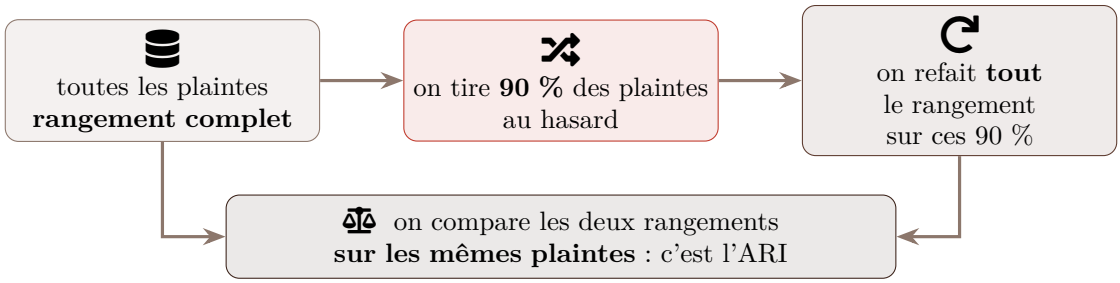
Et pour les incidents ? Un incident est une **zone dense** au milieu de plaintes variées : c'est pour ça que HDBSCAN est le premier choix. Ward n'est que le plan B, quand les plaintes ne forment pas d'îles nettes mais un **continuum**. Là, une méthode de densité ne trouve rien, ou un seul bloc, alors que Ward découpe quand même quelque chose d'exploitable, avec les mêmes règles d'admission derrière.

Donc Ward n'est pas « ajouté après HDBSCAN » : il est essayé après lui, **à sa place**, uniquement si HDBSCAN a échoué sur toute la grille. Dans l'exécution normale, **Ward ne tourne pas du tout**.

23. La stabilité et l'ARI, comme dans une classe (exercice 3.5)

23.1 Le test : refaire les équipes avec des absents

Une maîtresse range ses élèves en équipes selon leurs goûts. Un jour, quelques élèves sont absents : elle refait les équipes avec ceux qui restent. Si les amis qui étaient ensemble se retrouvent encore ensemble, et ceux qui étaient séparés restent séparés, sa façon de faire les équipes est **fiable**.
Le notebook fait pareil avec les plaintes :



Trois fois

On recommence trois fois. Chaque tirage repart de la base **complète** : on reste toujours à 90 %, on ne descend pas à 80 puis 70 %. Seules les plaintes tirées changent d'une fois à l'autre.

Un hasard fixé

Les tirages utilisent les graines 1, 2 et 3 : si on relance le notebook, on retombe sur les trois mêmes tirages, donc sur les mêmes résultats.

23.2 Comment on compte : des paires d'élèves

On ne compare pas les **numéros** des équipes : l'équipe « 1 » d'un jour peut s'appeler « 2 » le lendemain, ça ne veut rien dire. On regarde des **paires** : pour chaque paire d'élèves, les deux rangements sont-ils d'accord pour dire « ensemble » ou « séparés » ?

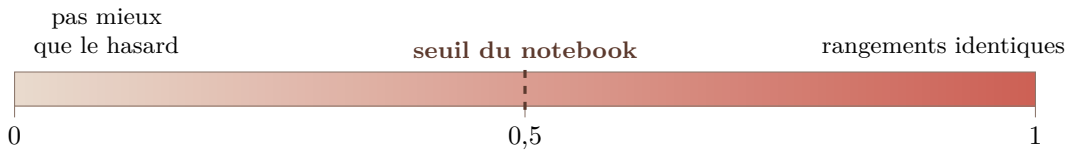


paire	lundi, mardi	
A et B	ensemble, ensemble	✓
A et C	séparés, séparés	✓
A et D	séparés, séparés	✓
B et C	séparés, séparés	✓
B et D	séparés, séparés	✓
C et D	ensemble, séparés	✗

Exemple inventé : 4 élèves, 6 paires ; les deux jours sont d'accord sur 5 paires sur 6.

23.3 Pourquoi « ajusté » ?

Compter les paires d'accord donne l'**indice de Rand**. Mais même deux rangements faits **au hasard** tombent souvent d'accord, surtout sur « séparés », parce que la plupart des paires sont séparées dans n'importe quel rangement. L'indice de Rand **ajusté** (ARI) retire cette part due au hasard :



Tu dois voir : trois valeurs d'ARI, une par tirage, toutes au-dessus de 0,5.

Stable ne veut pas dire juste. Mettre tout le monde dans une seule équipe donne un ARI parfait à chaque fois, et ne range rien. C'est pourquoi l'exercice 3.3 écarte d'abord les rangements ratés, et ne regarde la stabilité qu'ensuite.